quarry ui

DOCUMENTATION

Build with Quarry UI

Install, theme and customise free React components, templates, backgrounds and loaders. Everything you copy becomes your own code.

quarry-ui.vercel.app

MIT License

Contents

GETTING STARTED

01 Introduction

02 Installation

03 Quick start

04 Copy prompt

GUIDES

05 Theming

06 Components

07 Templates

08 Backgrounds

09 ASCII Aquarium

10 ASCII Lagoon

11 Loaders

12 Design systems

REFERENCE

13 Registry

14 Run it yourself

15 Contributing

16 Troubleshooting and FAQ

17 License and credits

Introduction

Quarry UI is a free, open source library of React components, full app templates, backgrounds and loaders. You copy what you need into your project and it becomes your code.

There is no package to install and no version to keep up with. Each item is a small registry entry that the shadcn CLI turns into plain files in your repository. Read them, change them, delete the parts you do not need.

What is inside

Collection	Count	What it is
Components	86	Sections and elements: footers, pricing, testimonials, navbars, galleries, forms, text effects and more.
Templates	4	Whole apps that install into your Next.js project: AI Chat, File Manager, Learning Platform and Abyss.
Backgrounds	13	Live canvases with controls, for hero sections and full pages.
Loaders	18	Small loading motions you can drop in anywhere.
Design systems	78	DESIGN.md briefs for well known brands, written for AI tools to read.

How it works

1. Find something on the site and open its page.
2. Run the install command, or press Copy prompt and paste it into your AI tool.
3. The CLI writes the source into your project and adds any packages it needs.
4. Import it, pass your own content, and change anything you like.

Principles

- **Your theme, not ours.** Components only use shadcn tokens, so they follow your colours and dark mode.
- **One file each.** Open a component and you can read the whole thing.
- **Motion that explains.** Things move to show what changed, never just to show off.
- **Accessible by default.** Real buttons and links, visible focus, and reduced motion respected.

Installation

Quarry UI installs through the shadcn CLI. If your project already uses shadcn/ui, you can add a component right now.

Requirements

- A React project. Templates need Next.js with the App Router.
- Tailwind CSS 4.
- shadcn/ui set up in the project, so a `components.json` file exists.
- Node.js 20 or newer.

Set up shadcn once

Skip this if you already have a `components.json`.

```
npx shadcn@latest init
```

Add a component

Every item has a JSON address on this site. Pass it to the CLI:

```
npx shadcn@latest add https://quarry-ui.vercel.app/r/mega-footer.json
```

The CLI copies the file into your components folder, installs the packages it needs (usually `framer-motion` and `lucide-react`) and makes sure `lib/utils.ts` has the `cn()` helper.

Other package managers

Manager	Command
npm	npx shadcn@latest add https://quarry-ui.vercel.app/r/<name>.json
pnpm	pnpm dlx shadcn@latest add https://quarry-ui.vercel.app/r/<name>.json
yarn	yarn dlx shadcn@latest add https://quarry-ui.vercel.app/r/<name>.json
bun	bunx --bun shadcn@latest add https://quarry-ui.vercel.app/r/<name>.json

Add a template

Templates are installed the same way. They write several files, including pages under `app/`:

```
npx shadcn@latest add https://quarry-ui.vercel.app/r/ai-chat.json
```

Start your dev server and open the route the template documents, for example `/ai-chat`.

Overwriting

If a file already exists, the CLI asks before replacing it. Commit your work first so you can review the diff.

Quick start

From an empty page to a working component in about a minute.

This walks through the Mega Footer. Every other component follows the same steps.

1. Install it

```
npx shadcn@latest add https://quarry-ui.vercel.app/r/mega-footer.json
```

2. Use it

```
import { MegaFooter } from "@/components/mega-footer";

export default function Page() {
  return <MegaFooter brand="Northwind" />;
}
```

3. Make it yours

Every component has sensible defaults and accepts your own content through props. Each one also takes a `className`, merged with `cn()`, so you can adjust spacing and size without editing the file.

```
<MegaFooter brand="Northwind" className="border-t" />
```

When props are not enough, open the file in `components/` and edit it. It is your code.

Each page shows its own usage

Open any item on the site to see a live preview, the install command, the props and a usage snippet.

Copy prompt

Prefer to let an AI tool do the work? Every item has a Copy prompt button.

The prompt contains everything the tool needs: what to install, how to import it, and for some tools the full source. Paste it into the tool and it sets the component up in your project.

Supported tools

Tool	How it works
Claude Code	Runs the install command for you
Codex	Runs the install command for you
Cursor	Agent mode with a terminal
v0	Paste the source
Lovable	Paste the source
bolt.new	Paste the source
Replit	Agent with a shell

Use the arrow next to the button to choose your tool. The site remembers your choice.

Limits

You can copy a couple of prompts without an account. After that, sign in with Google or GitHub to keep copying. Signing in is free and only stores your name, email and bookmarks. The install command and the source code are never limited.

Guides

Theming

Components follow your design system because they only use shadcn tokens.

There are no hard coded colours in a Quarry component. Everything reads CSS variables from your `globals.css`, so a new theme or dark mode applies to every component at once.

Tokens in use

Token	Used for
`background`, `foreground`	Page surface and default text
`card`, `card-foreground`	Raised panels and cards
`muted`, `muted-foreground`	Quiet surfaces and secondary text
`primary`, `primary-foreground`	Main buttons and emphasis
`accent`, `accent-foreground`	Hover states and highlights
`border`, `ring`	Outlines and focus rings
`destructive`	Errors and delete actions

Change the look

Edit the variables in your stylesheet. Nothing inside the components needs to change.

```
:root {
  --background: oklch(0.985 0 0);
  --foreground: oklch(0.16 0 0);
  --primary: oklch(0.18 0 0);
  --primary-foreground: oklch(0.985 0 0);
  --border: oklch(0.91 0 0);
}

.dark {
  --background: oklch(0.12 0 0);
  --foreground: oklch(0.97 0 0);
  --primary: oklch(0.97 0 0);
  --primary-foreground: oklch(0.14 0 0);
}
```

Dark mode

Use the `dark` class on a parent element, the same way shadcn does. Components switch with it and need no extra props.

Presets on the site

Previews on the site wrap some components in a preset, a small set of tokens and a font they were designed with. The component page shows the preset as CSS you can paste into your own `globals.css` if you want the same look.

Guides

Components

How components are built, and how to change them.

Anatomy

Each component is a single file with a named export and a `className` prop.

```
import { cn } from "@/lib/utils";

export function Example({ title = "Hello", className }: { title?: string; className?:
string }) {
  return <section className={cn("rounded-2xl border bg-card p-6", className)}>{title}
</section>;
}
```

Sections and elements

Sections are page sized: heroes, feature grids, pricing, footers. Elements are smaller: buttons, inputs, badges, cards. Use the sidebar to browse either.

Customising

- **Props first.** Pass your own text, items and images.
- **className second.** Adjust layout and spacing from outside.
- **Edit the source last.** The file is in your repository. Change what you like.

Motion

Motion uses `framer-motion` or plain CSS. Many ambient effects check `prefers-reduced-motion` and fall back to a still frame. Interactive motion keeps durations short.

Dependencies

The install command adds what a component needs. Most use only `framer-motion` and `lucide-react`. A few backgrounds use no packages at all.

Guides

Templates

Complete apps that install into a Next.js project and become yours.

A template is a set of files: pages under `app/`, components, a small data layer and a layout. Everything runs on sample data, so you can see it working before you connect anything.

Template	Install	Routes
AI Chat	npx shadcn@latest add https://quarry-ui.vercel.app/r/ai-chat.json	/ai-chat, /ai-chat/c/[id], /ai-chat/projects/[slug]
File Manager	npx shadcn@latest add https://quarry-ui.vercel.app/r/file-manager.json	/file-manager and its folders
Learning Platform	npx shadcn@latest add https://quarry-ui.vercel.app/r/lms.json	/lms, /lms/content, /lms/reports
Abyss	npx shadcn@latest add https://quarry-ui.vercel.app/r/abyss.json	/abyss, /abyss/login, /abyss/signup

Mount point

Each template lives under its own folder in `app/`. Links inside it work wherever it is mounted, so you can move the folder or serve it from a different path.

Connect your backend

- **AI Chat and Abyss.** Replace the `replyFor` function in the data file with a call to your model API and stream tokens into the message.
- **Abyss sign in and sign up.** The forms validate input and then open the chat. Connect their submit handlers to your auth service.
- **File Manager and Learning Platform.** Swap the sample data in the template's `lib` folder for your own queries.

Theme

Templates read your tokens. The preview on the site sets its own theme so you see the design as intended.

Guides

Backgrounds

Live canvases for hero sections and full pages, each with controls you can tune.

Backgrounds are components that fill their container. Give them a height, put your content inside as children, and they draw behind it.

```
import { FlowField } from "@/components/flow-field";

export default function Hero() {
  return (
    <div className="h-screen">
      <FlowField interaction="swirl">
        <h1>Your headline</h1>
      </FlowField>
    </div>
  );
}
```

Controls

The preview on every background page has controls for its props: colour, speed, density, cursor behaviour. Set them until it looks right, then copy the values into your code.

Performance

- Canvases pause when scrolled out of view.
- Reduced motion shows a still frame instead of animating.

Guides

ASCII Aquarium

Sea creatures, birds and pets drawn as live ASCII, with colourways and controls. Used by the Abyss template.

The aquarium paints each animal into a tiny hidden canvas, one pixel per character cell, then turns every pixel into a glyph by how dark it is. Large animals such as sharks and whales are drawn as soft shapes and look like shaded ASCII art. Small ones such as birds, fish, butterflies and pets are pixel sprites on muted tiles.

```
npx shadcn@latest add https://quarry-ui.vercel.app/r/ascii-aquarium.json
```

```
import { AsciiAquarium } from "@/components/ascii-aquarium";

export default function Hero() {
  return (
    <div className="h-screen">
      <AsciiAquarium animal="jellyfish" colorway="dusk">
        <h1>Something is swimming</h1>
      </AsciiAquarium>
    </div>
  );
}
```

Animals

Group	Values
Ocean	`shark` , `hammerhead` , `manta` , `whale` , `turtle` , `jellyfish` , `school`
Sky	`birds` , `butterflies`
Land	`dog` , `cat` , `rabbit` , `pets`

Land animals sit in a small meadow with trees and grass. They blink, wag, and hop to a new spot every few seconds. `pets` shows a dog, a cat and a rabbit together.

Colourways

`theme` reads your own `background`, `foreground` and `primary` tokens and follows light and dark mode. The others are fixed: `abyss`, `mono`, `ember`, `kelp`, `dusk` and `paper`. The `COLORWAYS` export lists their colours.

Props

Prop	Default	What it does
`animal`	`shark`	Which animal or scene to draw.
`colorway`	`theme`	Colours. `theme` uses your tokens.
`count`	auto	How many animals. Chosen from the stage size when empty.
`speed`	`1`	Pace of everything. Can be changed live.
`scale`	`1`	Size of the animals. Can be changed live.
`detail`	`9`	Size of one character cell in px. Smaller is finer.
`look`	`auto`	`tiles`, `glyphs` or `auto`, which picks what suits the animal.
`grid`	`true`	Faint cell grid behind everything. Tiles only.
`bubbles`	`true`	Rising flecks in water scenes.
`pointer`	`avoid`	`avoid`, `attract` or `off`. How animals react to the cursor.
`paused`	`false`	Stops the motion and keeps the frame.
`calm`	`0.6`	Fades the scene behind your content, 0 to 1.
`dim`	`0`	Fades the whole scene, 0 to 1.
`avoid`	children	A ref to an element animals should keep clear of.
`className`	none	Set a height here. The component fills its container.

Tips

- Always give the container a height. The scene fills it.
- Use `avoid` with a ref to your prompt box or card so animals swim around it.
- Raise `dim` behind long text, and lower `speed` for calmer screens.
- Reduced motion draws a single still frame and stops the loop.

Guides

ASCII Lagoon

A top down Hawaiian lagoon drawn as ASCII cells, on a seamless loop.

Turtles, a manta, dolphins, a school of yellow tangs and a humpback swim over a reef while hibiscus and plumeria drift past. Sun bands and caustics move across the water. The whole scene is a pure function of time, and every motion has a whole number of cycles per loop, so the 24.5 second loop repeats without a seam.

```
npx shadcn@latest add https://quarry-ui.vercel.app/r/ascii-lagoon.json
```

```
import { AsciiLagoon } from "@/components/ascii-lagoon";

export default function Hero() {
  return (
    <div className="h-screen">
      <AsciiLagoon>
        <h1>What is on your mind?</h1>
      </AsciiLagoon>
    </div>
  );
}
```

Props

Prop	Default	What it does
`speed`	`1`	Playback speed. Can be changed live without a jump.
`glyphs`	`true`	Draw a character on every cell. Off leaves plain coloured cells.
`frost`	`true`	Blur and brighten the water behind the children.
`paused`	`false`	Hold the current frame.
`className`	none	Set a height here. The scene covers its container.

Notes

- The scene is drawn on a fixed 1944 by 1760 stage and scaled to cover its container, so it fits any shape.
- The palette is fixed. It is a scene, not a theme. Put dark text on it, as the frosted centre is bright.
- It uses your Geist Mono font when the page has it and falls back to a system monospace.
- It pauses when scrolled out of view. Reduced motion shows one still frame.
- To record a video, screen record the tab. One loop is 24.5 seconds.

Loaders

Small loading motions that match any theme.

Loaders are tiny components with no required props. Drop one where something is loading.

```
import { ArcSpinner } from "@/components/arc-spinner";

<ArcSpinner className="size-8" />
```

They use `currentColor`, so they take the text colour of their parent. Put them in a button, a card or a full page overlay.

Design systems

DESIGN.md briefs your AI tool can read to build in a familiar visual language.

A DESIGN.md is a plain text description of a design language: colours, type, spacing, components and the reasoning behind them. The site has briefs for 78 well known brands.

How to use one

1. Open a brand on the Design systems page and download or copy its DESIGN.md.
2. Save it in your project root as `DESIGN.md`.
3. Ask your AI tool to build a page following it.

```
Build a pricing page that follows DESIGN.md. Use its colours, type scale and component rules.
```

Brand names

The briefs describe the public look of each brand for reference. Names and designs belong to their owners, and a brief gives you no right to use their trademarks. Build your own product in your own name.

Reference

Registry

How items are published, and how tools and agents can read them.

Quarry UI is a shadcn registry. Every item is a JSON file at a predictable address.

```
https://quarry-ui.vercel.app/r/<name>.json
```

Machine readable indexes

Address	Contents
`/llms.txt`	A short index of every component, template, background and loader.
`/llms-full.txt`	The index with descriptions and usage for each item.
`/sitemap.xml`	Every public page.
`/r/<name>.json`	The install payload for one item.

Item shape

```
{
  "name": "mega-footer",
  "type": "registry:component",
  "title": "Mega Footer",
  "dependencies": ["framer-motion"],
  "registryDependencies": ["utils"],
  "files": [{ "path": "registry/quarry/mega-footer.tsx", "type": "registry:component"
}]
}
```

Multi file items, such as templates, list every file with a `target` path that says where it is written in your project.

Reference

Run it yourself

Clone the site, run it locally, or deploy your own copy.

Run locally

```
git clone https://github.com/PRODHOSH/quarry-ui.git
cd quarry-ui
npm install
npm run dev
```

Open http://localhost:3000. The site works without any configuration. Sign in, bookmarks and the waiting list need a Supabase project.

Scripts

Command	What it does
`npm run dev`	Start the dev server.
`npm run build`	Build the registry JSON, then the site.
`npm run registry:build`	Build only the registry JSON into `public/r`.
`npm run lint`	Run ESLint.
`npm run typecheck`	Run the TypeScript compiler without emitting.

Environment variables

Copy `.env.example` to `.env.local` and fill what you need.

Variable	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Your Supabase project URL.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	The public anon key from Project settings, API.
`NEXT_PUBLIC_SITE_URL`	The public address of your deployment. Used in links, the sitemap and install commands.
`NEXT_PUBLIC_CONTACT_EMAIL`	Where the contact links point.
`NEXT_PUBLIC_GITHUB_URL`	Your repository address.
`PROMPT_SECRET`	Server only. A long random string that signs the free prompt counter. Required in production. Create one with `openssl rand -hex 32`.
`NEXT_PUBLIC_GA_ID`	Google Analytics measurement id. Leave empty to turn it off.

Set up Supabase

1. Create a free project at supabase.com.
2. Run the files in `supabase/migrations` in order: `0001_bookmarks.sql`, `0002_waitlist.sql`, `0003_prompt_usage.sql`.
3. Under Authentication, turn on the Google and GitHub providers and add your keys.
4. Under Authentication, URL configuration, set the Site URL to your address and add it to the redirect list.
5. Copy the project URL and anon key into `.env.local`.

Bookmarks and the prompt counter are protected by row level security, so people can only read their own rows.

Deploy

Any host that runs Next.js works. On Vercel, import the repository, add the environment variables and deploy. The build command already runs the registry build first.

Reference

Contributing

One good pull request makes a real difference. Here is the short version.

Add a component

1. Create `registry/quarry/<name>.tsx` with a named export and a `className` prop.
2. Add an entry to `registry.json`. Copy a neighbour and change the details.
3. Add a preview in `registry/demos.tsx`.
4. Run `npm run registry:build`.
5. Open the page, then check light and dark mode, phone width and the keyboard.
6. Run `npm run lint` and `npm run typecheck`, then open the pull request.

What a good component looks like

- Theme tokens only. No hex values and no palette colours.
- One self contained file that imports only `@/lib/utils`, `framer-motion` and `lucide-react`.
- A named export and a `className` prop merged with `cn()`.
- Looks good with zero props, and the content is overridable.
- Short, purposeful motion that respects reduced motion.
- Accessible: real buttons, visible focus, labels on icon buttons.
- Works at phone width.

The full guide is in CONTRIBUTING.md in the repository. Security problems should be reported privately, as described in SECURITY.md.

Reference

Troubleshooting and FAQ

Quick answers to the things people ask most.

The install command says components.json is missing

shadcn is not set up in the project yet. Run `npx shadcn@latest init`, then try again.

A component looks unstyled

Check that Tailwind CSS 4 is installed and that your stylesheet defines the shadcn tokens. Components read `--background`, `--foreground`, `--primary` and the rest, so a missing token shows as missing colour.

I get an error about the utils file

Components import `cn` from `@/lib/utils`. The CLI creates it for you. If you moved it, update the import or restore the file.

My background shows nothing

The container needs a height. Add `h-screen` or a fixed height to the parent, or pass a height class through `className`.

Can I use it in a commercial project?

Yes. The code is MIT licensed. No credit is needed.

Do I need an account?

No. Browsing, installing and copying source are open. An account is only needed for bookmarks and for copying more than a couple of prompts.

Does it work with Vite or Remix?

Components do, as long as you have Tailwind CSS 4 and shadcn set up. Templates are built for the Next.js App Router.

How do I get updates?

Run the install command again for the item and review the diff. Because the code is yours, the CLI asks before replacing files you have changed.

I found a bug or want a component

Open an issue on GitHub, or write to hello@prodhosh.me.

Reference

License and credits

Free to use anywhere.

Quarry UI is released under the MIT License. You can use, copy, change and distribute it, in private and commercial work, without asking and without credit. The software comes with no warranty.

A few assets come from other people and keep their own terms, including illustrations and the DESIGN.md collection. They are credited in THIRD_PARTY_NOTICES.md in the repository.

Brand names and designs mentioned in the design briefs belong to their owners.